



دانشکده مهندسی برق

مدارهای منطقی و سیستم‌های دیجیتال

آزمایش ۴ - طراحی، پیاده‌سازی و
تست مدارهای ترکیبی بر روی FPGA

آزمایش ۴ - طراحی ، پیاده سازی و تست مدار های ترکیبی بر روی FPGA

در آزمایش قبل با نحوه توصیف مدارات ترکیبی به زبان Verilog ، نرم افزار Vivado و چگونگی پیاده سازی مدار توصیف شده بر روی برد FPGA آشنا شدید. در این آزمایش، ابتدا به راه اندازی 7-segment پرداخته و سپس به پیاده سازی یک adder و نمایش خروجی بر روی 7-segment ها خواهیم پرداخت. در نهایت نیز با یکی از الگوریتم های کدینگ به نام hamming-code آشنا شده و یک encoder و decoder بر مبنای آن طراحی خواهیم کرد.

قطعات و تجهیزات مورد نیاز

- برد Zynq (AXPZ7010)
- Extension board
- یک عدد کابل USB و سیم پاور برد
- نرم افزار Vivado 2019.1

۱- پیش مطالعه

در این بخش با نحوه استفاده از 7-seg 4 رقمی و extension board و مفاهیم Hamming code آشنا خواهید شد. **! خواندن پیش مطالعه برای انجام آزمایش بسیار ضروری است!**

• 4-digit 7-seg :

در این قسمت به راه اندازی 7-seg چهار رقمی می پردازیم که از قرار زیر می باشد :



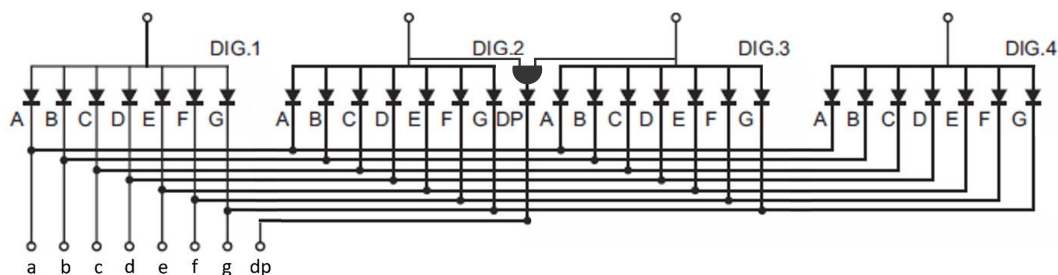
شکل ۴-۱: 7-seg چهار رقمی

این قطعه ۱۲ پایه دارد :

۷ پایه از آن به LED های a تا g هر چهار 7-seg متصل است. ۱ پایه به decimal (dp point) که در اینجا همان ":" است، متصل است. ۴ پایه به dig0, dig1, dig2, dig3 وصل اند که با یک شدن، آن رقم را روشن می کنند.

مدلی از 7-seg که ما در این آزمایش استفاده می کنیم، دارای مشخصات زیر است. دقت داشته باشید که مدل های مختلفی وجود دارند و باید برای آگاهی از این موارد ، در صورت وجود دیتاشیت، به آن مراجعه کنیم و در غیر این صورت از تست دیود مولتی متر کمک بگیریم:

پایه های a تا g و dp "Active LOW" هستند. پایه های dig0 تا dig3 "Active HIGH" هستند.



شکل ۴-۲: مدار داخلی یک 7-seg چهار رقمی

با توجه به مدار فوق، مشاهده می‌کنیم که اگر هر یک از Dig ها یک باشد و A-G و dp صفر، جریان به سمت هر یک از LED های آن سگمنت، شروع به حرکت می‌کند و آن را روشن می‌کند که نشان دهنده علت "Active HIGH" بودن Dig ها و "Active LOW" بودن dp و پایه های A-G است. توجه داشته باشید که در این مدل از 7-seg ها، تنها هنگامی قادر به روشن کردن dp هستیم که dig چپ و راست dp روشن شده باشد. (همانند یک ساعت) در تصاویر زیر، دو حالت از نمایش اعداد را مشاهده می‌کنید. توصیه می‌شود که برای اینکه "Active LOW" بودن LED ها شما را دچار اشتباه نکند، ابتدا با فرض اینکه عدد مورد نظر "Active HIGH" است، آن را نوشته و سپس قرینه کنید:



$$\{\text{dig3}, \text{dig2}, \text{dig1}, \text{dig0}\} = 0110$$

$$\{a, b, c, d, e, f, g\} = 0100100 = \sim(1011011)$$

$$\text{dp} = 0 = \sim(1)$$



$$\{\text{dig3}, \text{dig2}, \text{dig1}, \text{dig0}\} = 0001$$

$$\{a, b, c, d, e, f, g\} = 1001111 = \sim(0110000)$$

$$\text{dp} = 1 = \sim(0)$$

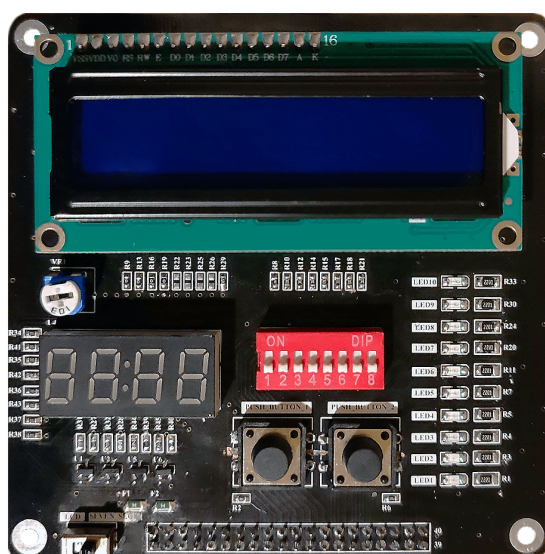
شکل ۴-۳: دو مثال از نمایش اعداد روی 7-seg چهار رقمی

• Extension board :

برای پیاده سازی مداراتی که تعداد ورودی و خروجی زیادی دارند، نیازمند تعداد بالایی از کلید و LED ها هستیم. بردی که در دسترس شما قرار دارد، بردی صنعتی است و از تعداد کلید و LED کمتری نسبت به برد های آموزشی یا به اصطلاح Evaluation board برخوردار است. برای حل این مسئله و امکان پیاده سازی مدارات پیشرفته تر، یک برد کمکی برای شما فراهم شده است که به برد قابلیت دسترسی به پیرفرال های زیر را اضافه می کند:

- 10 LEDs
- 8 Dip switches
- 2 Push buttons
- 4-digit 7-seg
- LCD display

برد اصلی به صورت hardwired به برخی از پایه های تراشه زینک متصل شده است (مثلا 5 LED موجود بر روی برد اصلی) و تعدادی از پین های آن، توسط شرکت سازنده آزاد قرار داده شده است تا کاربر بسته به نیاز خود از آن ها استفاده کند. دو مجموعه expansion port با مجموعاً ۸۰ پین، در دو سمت برد تعبیه شده است که می تواند به عنوان خروجی یا ورودی به هر پیرفرالی که مد نظر ما باشد متصل شود.



شکل ۴-۴: نمایی از برد کمکی

در این جدول، اتصال پین های برد کمکی به پین های تراشه زینک را مشاهده می کنید:
(تمامی اتصالات در این **constraint file** نیز موجود است)

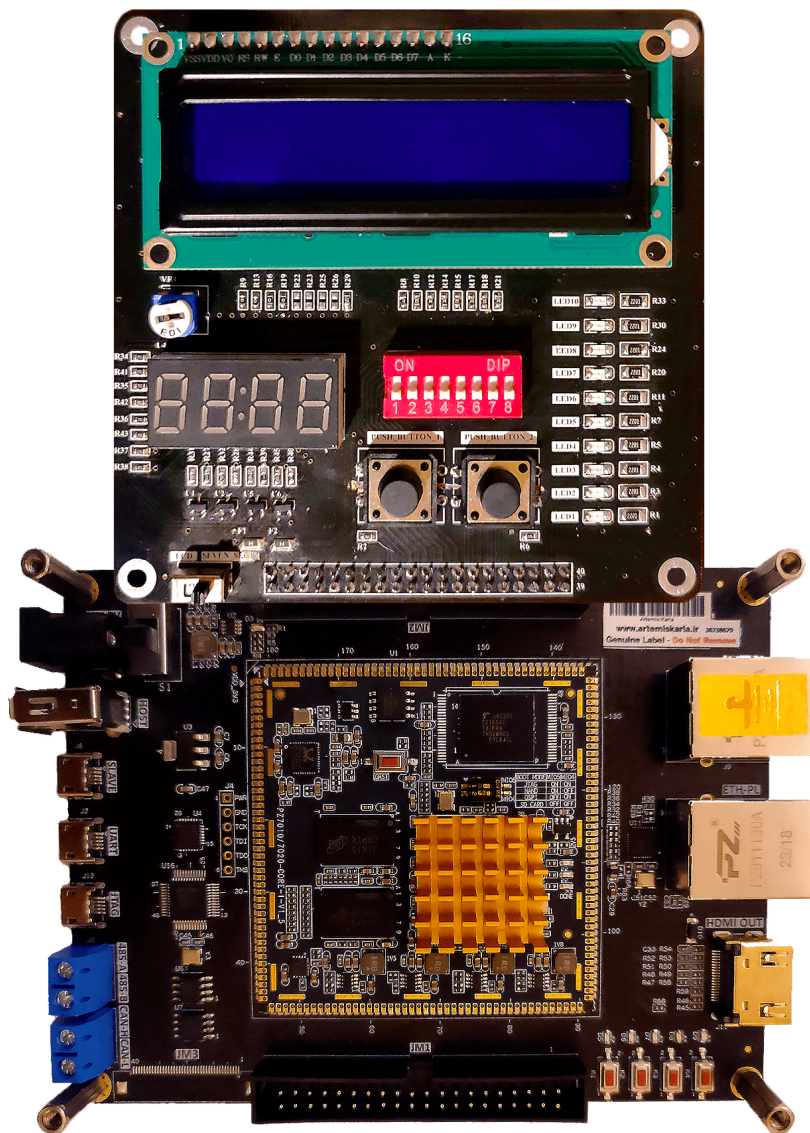
Pinout					
Id.	Ext. board	Zynq	Id.	Ext. board	Zynq
1	Vdd 5V	Vdd 5V	21	Vdd 3.3V	Vdd 3.3V
2	GND	GND	22	GND	GND
3	7-seg "dig3" LCD "RS"	B19	23	7-seg "dig2" LCD "RW"	G20
4	7-seg "dig1" LCD "EN"	D19	24	7-seg "dig0"	D20
5	7-seg "a" LCD "D0"	F16	25	7-seg "b" LCD "D1"	F17
6	7-seg "c" LCD "D2"	H15	26	7-seg "d" LCD "D3"	G15
7	7-seg "e" LCD "D4"	J18	27	7-seg "f" LCD "D5"	H18
8	7-seg "g" LCD "D6"	H16	28	7-seg "dp" LCD "D7"	H17
9	Push button1	J20	29	Push button2	H20
10	DIP 1	K14	30	DIP 2	J14
11	DIP 3	M14	31	DIP 4	M15
12	DIP 5	L19	32	DIP 6	L20
13	DIP 7	M19	33	DIP 8	M20
14	LED 10	N15	34	LED 9	N16
15	LED 8	C20	35	LED 7	B20
16	LED 6	E17	36	LED 5	D18
17	GND	GND	37	GND	GND
18	GND	GND	38	GND	GND
19	LED 4	E18	39	LED 3	E19
20	LED 2	G17	40	LED 1	G18

شکل ۴-۵: اتصالات پین های برد کمکی به پین های تراشه زینک

⚠ نکات مهم :

- LED های ۱ تا ۱۰، push button ها و dip switch ها همگی "Active HIGH" هستند.
- Dig3-Dig0 "Active HIGH" هستند اما g-a و dp همگی "Active LOW" هستند.

- سوییچی که در گوشه پایین و سمت چپ برد قرار دارد، همواره در مود 7-seg باشد.
- برد کمکی برای اتصال صحیح می بایست تنها و تنها از طریق پین های JM2 روی برد اصلی نصب شود.
- پین های نوشته شده در جدول خصوصاً vdd 5V و vdd 3.3v ، صرفاً pinout برای ۴۰ پین برد کمکی و پورت jm2 برد اصلی است و ارتباطی به I/O standard ای که در constraint فایل ست می کنید، ندارد. دقت داشته باشید که I/O standard تمامی پین ها LVCMOS33 می باشد.
- توجه داشته باشید که پین های Dip switch ها از چپ به راست شماره گذاری شده است. این در حالی است که ما به LSB اندیس ۰ و به MSB اندیس n را نسبت می دهیم که برعکس چینش ذکر شده است. فلذا برای اینکه کار کردن با switch ها راحت تر باشد، پین ها را برعکس تنظیم کنید. مثلاً برای ست کردن $A[7:0]$ ، $A[7]$ را به Dip1 ، $A[6]$ را به Dip2 و ... وصل کنید. البته این نکته در فایل constraint داده شده رعایت شده است. چنانچه خودتان تمایل به تنظیم آن ها دارید، به این نکته دقت داشته باشید.



شکل ۴-۶: برد کمکی نصب شده بر روی برد اصلی

Hamming code :

در بخشی از آزمایش باید مدار Error Correcting Code (ECC) را به روش [7, 4, 3] Hamming بسازید. فرض کنید قصد داریم چهار بیت را از محل A به محل B ارسال کنیم. با توجه به اینکه ممکن است در مسیر ارسال یکی از بیت ها تغییر کند (نویز یا هر عامل مزاحم) ، سه بیت اضافی به همراه دیتای اصلی به B ارسال می شود. این سه بیت حاوی اطلاعاتی است که در صورت تغییر یکی از داده ها، B می تواند آن را تشخیص و سپس محل آن را شناسایی کرده و در نتیجه دیتا را بازیابی نماید. به این سه بیت، بیت های parity (در فارسی: بیت برابری یا توازن) گفته می شود. برای پیاده سازی این روش نیازمند دو مدار هستیم. یک مدار فرستنده در محل A که بیت های parity را تولید می کند، و یک مدار گیرنده در محل B که با توجه به هفت بیت دریافتی، بیت معیوب را شناسایی و کد ارسالی را بازسازی می کند.

برای اطلاعات بیشتر در مورد Hamming code و به صورت کلی و [7, 4, 3] Hamming که مورد بحث این آزمایش است به ترتیب به [این ویدیو](#) و [این سایت](#) مراجعه کنید.

۲- پیش‌گزارش

- پیش مطالعه را بخوانید و با نحوه کارکرد بورد کمکی ، 7-seg و hamming code آشنا شوید.
- متن دستور کار را مطالعه کرده و کد وریلاگ مورد نیاز تمامی بخش ها را بنویسید.
(دقت داشته باشید که در شروع آزمایش، موجود بودن کد بخش های مختلف توسط دستیاران بررسی می شود و تنها دانشجویانی که کد ها را زده باشند اجازه انجام آزمایش را خواهند داشت.)

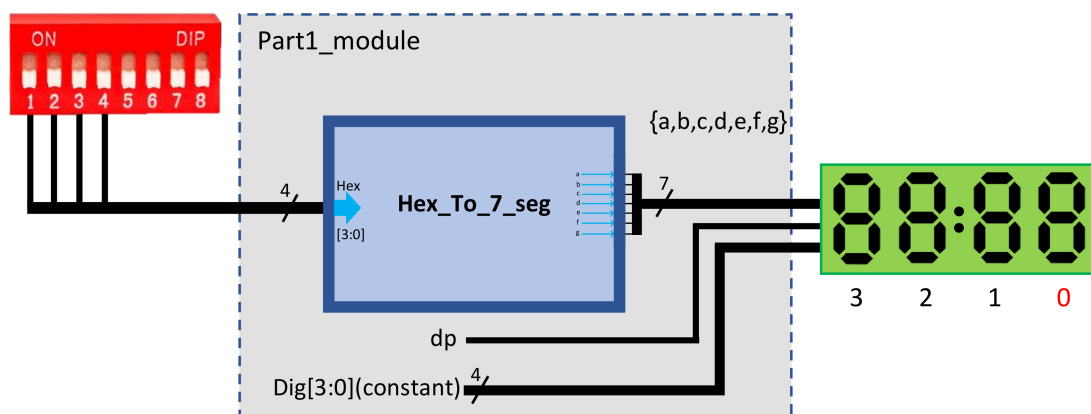
۳- دستور کار

۳-۱ بخش اول (زمان تقریبی: ۱۵ دقیقه)

در این بخش به نحوه ی استفاده از 7-seg مسلط می شویم تا در بخش های بعدی از آن استفاده کنیم. ماژولی بنویسید که با instance گرفتن از ماژول Hex_to_7_seg ، در ورودی عددی ۴ بیتی (از dip switch ها) را گرفته و آن را در قالب عدد hex بر روی dig0 نمایش دهد. یعنی عدد ۱۰ تا ۱۵ را به صورت A تا F نمایش دهد.

برای سهولت کار، یک ماژول Hex_to_7_seg به شما داده شده است که در ورودی آن یک عدد ۴ بیتی گرفته می شود و در خروجی، پترن مورد نیاز برای LED های a تا g سون سگمنت جهت نمایش عدد مورد نظر قرار می گیرد.

در نهایت مدار شما می بایست به شکل زیر باشد :



شکل ۴-۷: شماتیک مدار بخش اول

⚠ در این بخش و دیگر بخش ها، صرفاً ماژول Hex_to_7_seg را instantiate کرده و

هیچ تغییری در تعریف ماژول اعمال نکنید!

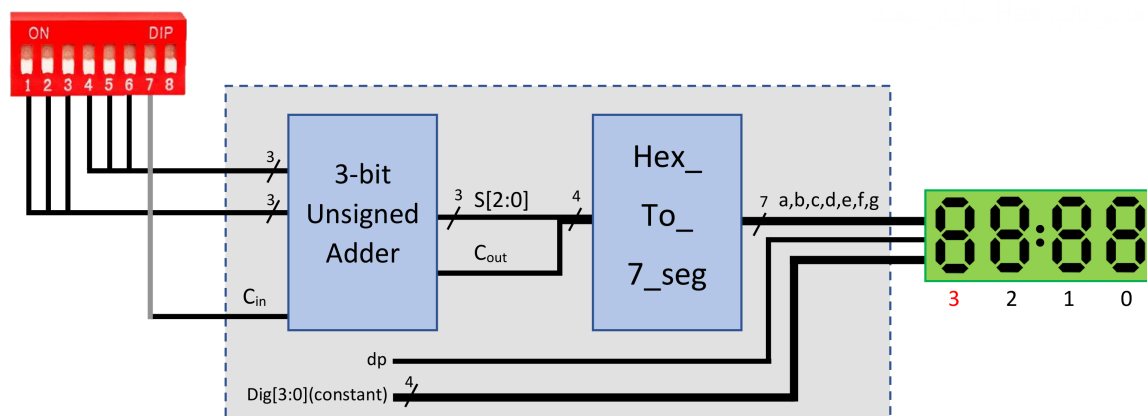
```
module Hex_to_7_seg
(
    input wire [3:0] Hex,
    output reg a,b,c,d,e,f,g
);

always @(Hex) begin
    case(Hex)
        0 : {a,b,c,d,e,f,g} = ~(7'b1111110);
        1 : {a,b,c,d,e,f,g} = ~(7'b0110000);
        2 : {a,b,c,d,e,f,g} = ~(7'b1101101);
        3 : {a,b,c,d,e,f,g} = ~(7'b1111001);
        4 : {a,b,c,d,e,f,g} = ~(7'b0110011);
        5 : {a,b,c,d,e,f,g} = ~(7'b1011011);
        6 : {a,b,c,d,e,f,g} = ~(7'b1011111);
        7 : {a,b,c,d,e,f,g} = ~(7'b1110000);
        8 : {a,b,c,d,e,f,g} = ~(7'b1111111);
        9 : {a,b,c,d,e,f,g} = ~(7'b1111011);
        10 : {a,b,c,d,e,f,g} = ~(7'b1110111); // A
        11 : {a,b,c,d,e,f,g} = ~(7'b0011111); // b
        12 : {a,b,c,d,e,f,g} = ~(7'b1001110); // C
        13 : {a,b,c,d,e,f,g} = ~(7'b0111101); // d
        14 : {a,b,c,d,e,f,g} = ~(7'b1001111); // E
        15 : {a,b,c,d,e,f,g} = ~(7'b1000111); // F
    endcase
end

endmodule
```

۲-۳ بخش دوم (زمان تقریبی: ۳۰ دقیقه)

در آزمایش ۳ ماژول half adder و full adder را پیاده سازی کردید. حال با گرفتن از این دو ماژول یک unsigned adder سه بیتی طراحی کنید. مدار بایستی یک ورودی را از Dip3-Dip1، ورودی دیگر را از Dip6-Dip4 و carry in را از Dip7 بگیرد. حاصل جمع چهار بیتی (سه بیت خروجی جمع کننده ها به همراه بیت carry out) را بر Dig3 در قالب Hex نمایش دهید.

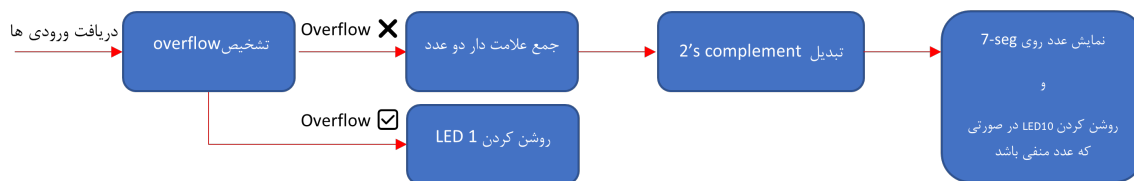


شکل ۴-۸: شماتیک مدار بخش دوم

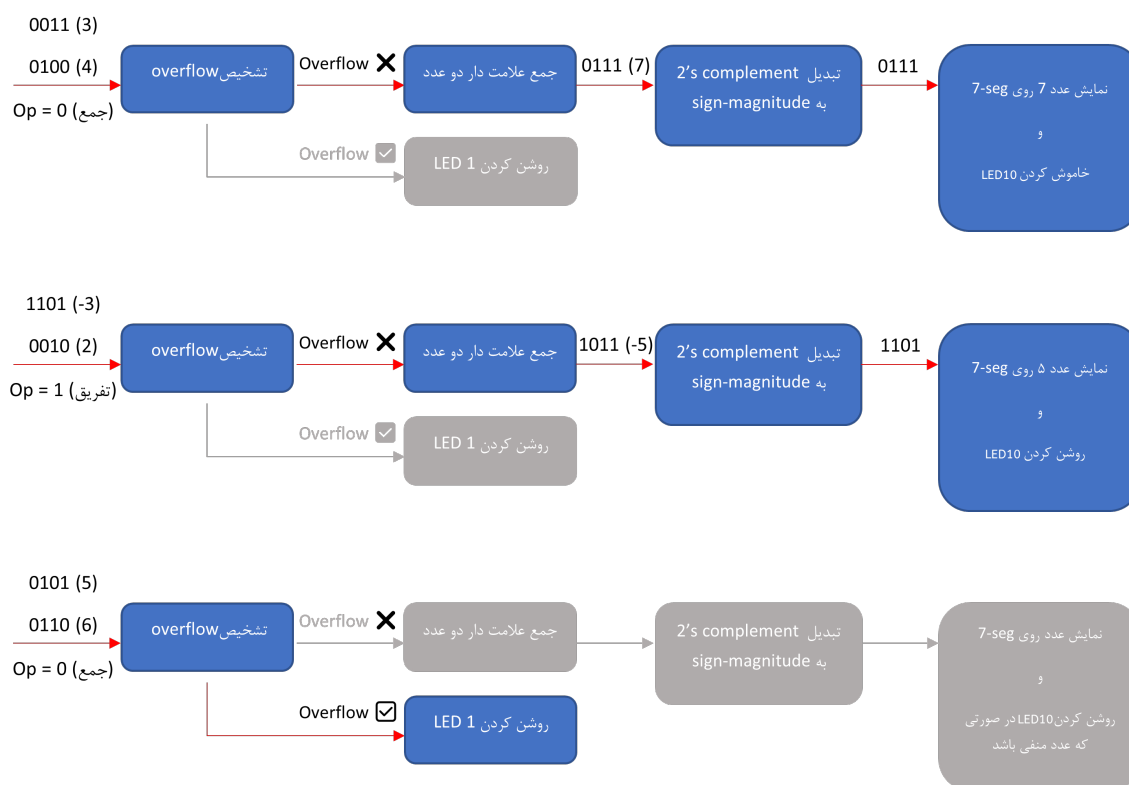
۳-۳ بخش سوم (زمان تقریبی: ۲۵ دقیقه)

حال با بکارگیری چهار full adder و مدارهای مورد نیاز دیگر، یک ماژول جمع و تفریق کننده ی چهار بیتی علامت دار طراحی کنید. (carry in را به صورت hardwired به صفر وصل کنید تا صرفاً دو عدد ۴ بیتی علامت دار در ورودی قرار گیرد). یک بیت op برای مدار خود در نظر بگیرید که با صفر بودن آن، عملیات جمع و با یک بودن آن، عملیات تفریق انجام شود. همچنین مدار تشخیص سرریز (overflow) را به آن اضافه کنید. چنانچه حاصل سرریز داشت، از آنجا که نمایش ۴ بیت جواب حاصل به صورت Hex بی معنا است، تنها LED1 برد کمکی را به نشانه وقوع این پدیده روشن کنید. در صورتی که سرریز رخ نداد، حاصل چهار بیتی علامت دار خروجی را پس از تبدیل به فرمت sign-magnitude بر روی Dig0 از سون سگمنت ها نشان دهید. برای نمایش علامت منفی، از LED10 استفاده کنید.

برای فهم بهتر از روند دریافت ورودی/خروجی و چند مثال، شکل ۴-۹ و ۴-۱۰ را ملاحظه کنید.



شکل ۴-۹: روند دریافت ورودی ها و نمایش خروجی



شکل ۴-۱۰: چند مثال ورودی و خروجی دریافتی مدار بخش سوم

ورودی ها و خروجی ها:

- دو ورودی چهار بیتی را به کمک Dip4-Dip1 و Dip8-Dip5 اعمال کنید.
- بیت op را به کمک push button1 مقداردهی کنید.
- خروجی را بر روی 7-seg Dig0 نمایش داده و در صورت منفی بودن LED10 را روشن نمایید.
- در صورت رخداد overflow ، LED1 از بورد کمکی را روشن کنید.

مقدمه بخش چهارم تا ششم :

با خوانش [بخش پیش مطالعه](#)، با مفهوم Hamming Code آشنا شدید. در بخش چهارم تا ششم می خواهیم خواهیم با ساز و کار مدار زیر آشنا شویم. در ابتدا به روش hamming، ۳ بیت parity برای داده ۴ بیتی ورودی محاسبه و به آن اضافه می شود و عددی ۷ بیتی از ماژول encoder خارج می شود. سپس با تشکیل ماژول transmission line، یکی از این ۷ بیت را قرینه می کنیم تا تاثیر نویز بر سیگنال را شبیه سازی کنیم. در نهایت داده ۷ بیتی نویزی را به ماژول decoder می دهیم تا بیت خراب را پیدا و اصلاح کند. سپس ۴ بیت داده را استخراج و در خروجی خود تحویل دهد :



شکل ۴-۱۱: مدار یک سیستم انکودر و دیکودر hamming

۴-۳ بخش چهارم (Encoder) (زمان تقریبی: ۳۰ دقیقه)



شکل ۴-۱۲: مدار انکودر

برای تولید بیت‌های parity و اضافه کردن آنها به ورودی چهار بیتی $x_0x_1x_2x_3$ باید به شکل زیر عمل نمود:

Encode: $x_0x_1x_2x_3 \rightarrow p_0p_1x_0p_2x_1x_2x_3$, where

$$p_0 = x_0 \oplus x_1 \oplus x_3,$$

$$p_1 = x_0 \oplus x_2 \oplus x_3,$$

$$p_2 = x_1 \oplus x_2 \oplus x_3.$$

مدار انکودر را طراحی و بر روی برد پیاده‌سازی کنید. برای ورودی از چهار کلید Dip4-Dip1 استفاده کنید و کد هفت بیتی حاصل را بر روی LED7-LED1 برد کمکی نمایش دهید. سپس به ازای تمامی ترکیب‌های ۱۶ گانه ورودی، هفت بیت خروجی را با جدول زیر مقایسه کنید. (بیت‌های قرمز رنگ parity bit ها و بیت‌های سیاه رنگ بیت‌های دیتا هستند)

Input	Output	Input	Output	Input	Output	Input	Output
0000	0000000	0100	1001100	1000	1110000	1100	0111100
0001	1101001	0101	0100101	1001	0011001	1101	1010101
0010	0101010	0110	1100110	1010	1011010	1110	0010110
0011	1000011	0111	0001111	1011	0110011	1111	1111111

شکل ۴-۱۳: جدول ورودی و خروجی‌های انکودر

۳-۵ بخش پنجم (Decoder) (زمان تقریبی: ۳۰ دقیقه)



شکل ۴-۱۴ مدار انکودر hamming

در بخش قبل encoder ای طراحی کردید که قادر به اضافه کردن بیت های parity به ۴ بیت داده ی ورودی بود. حال در این بخش فرض می کنیم که دیتای نویز دار به دست ما رسیده است و ما می خواهیم با ساخت یک decoder بیت خراب را شناسایی کنیم. سپس بیت خراب را اصلاح کرده و در خروجی ۴ بیت دیتای اصلی را استخراج کرده و همچنین در ۳ بیت، شماره بیت خراب را نمایش دهیم. شما به صورت دستی ۷ بیت داده دریافت شده را بر روی Dip7-Dip1 تنظیم می کنید و سپس ماژول شما می بایست این ۷ بیت را به صورت ورودی دریافت کند و بررسی کند که با توجه به بیت های parity ست شده در ۷ بیت دریافتی آیا خطایی در بیتی رخ داده یا خیر. چنانچه خطایی وجود داشت، آن بیت را اصلاح کند و ۴ بیت دیتا را بر روی LED4-LED1 مورد کمکی نمایش دهد. چنانچه داده ورودی خطایی نداشت، عدد ۰ را بر روی Dig0 سون سگمنت ها نمایش دهید و چنانچه بیت شماره n ام دچار خطا شده بود، عدد n را نمایش دهید.

Decode: Say we receive $p_0'p_1'x_0'p_2'x_1'x_2'x_3'$.

$(p_i' = p_i, x_j' = x_j \text{ if no errors})$

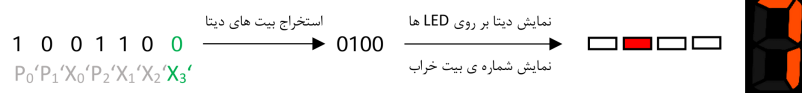
Let $c_0 = p_0' \oplus x_0' \oplus x_1' \oplus x_3'$,
 $c_1 = p_1' \oplus x_0' \oplus x_2' \oplus x_3'$,
 $c_2 = p_2' \oplus x_1' \oplus x_2' \oplus x_3'$.

$(c_2c_1c_0)_{10}$	Decode to
3	$\overline{x_0'}x_1'x_2'x_3'$
5	$x_0'\overline{x_1'}x_2'x_3'$
6	$x_0'x_1'\overline{x_2'}x_3'$
7	$x_0'x_1'x_2'\overline{x_3'}$
other values	$x_0'x_1'x_2'x_3'$

ترکیب $c_2c_1c_0$ محل بیت احتمالاً خراب را نشان می‌دهد. اگر هیچ خطایی رخ نداده باشد، بر اساس روابط بالا، این مقدار صفر خواهد بود، اما در صورت خرابی حداکثر یکی از بیت‌ها، محل آن توسط این عدد نمایش داده می‌شود. از آنجایی که برای ما تنها بیت‌های چهارگانه‌ی اصلی اهمیت دارند، باید طبق جدول مقابل، چهار بیت اصلی ارسالی از طرف فرستنده را استخراج کرده و بیت خراب را complement کرد. در مورد بقیه‌ی مقادیر عدد $c_2c_1c_0$ یعنی (۱، ۲، ۴)، خرابی در بیت parity رخ داده است. این بیت‌ها اگرچه در تشخیص خطا به ما کمک کرده اند، ولی به عنوان بخشی از پیام ارسالی نبوده و لذا در جدول به ازای این مقادیر، تغییری در کد دریافتی اعمال نمی‌شود. مجدداً یادآوری می‌شود صفر بودن این مقدار به معنای عدم وقوع هرگونه خطایی است.

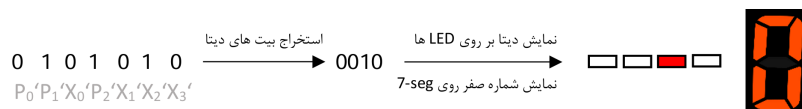
$$\begin{array}{l}
 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \\
 P_0'P_1'X_0'P_2'X_1'X_2'X_3'
 \end{array}
 \longrightarrow
 \left\{
 \begin{array}{l}
 C_0 = P_0' \wedge X_0' \wedge X_1' \wedge X_3' = 1 \wedge 0 \wedge 1 \wedge 1 = 1 \\
 C_1 = P_1' \wedge X_0' \wedge X_2' \wedge X_3' = 0 \wedge 0 \wedge 0 \wedge 1 = 1 \\
 C_2 = P_2' \wedge X_1' \wedge X_2' \wedge X_3' = 1 \wedge 1 \wedge 0 \wedge 1 = 1
 \end{array}
 \right\}
 \longrightarrow (C_2C_1C_0)_{10} = 7
 \longrightarrow$$

بیت ۷ام یا همان X_3' دچار خطا شده است.



$$\begin{array}{l}
 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \\
 P_0'P_1'X_0'P_2'X_1'X_2'X_3'
 \end{array}
 \longrightarrow
 \left\{
 \begin{array}{l}
 C_0 = P_0' \wedge X_0' \wedge X_1' \wedge X_3' = 0 \wedge 0 \wedge 0 \wedge 0 = 0 \\
 C_1 = P_1' \wedge X_0' \wedge X_2' \wedge X_3' = 0 \wedge 0 \wedge 0 \wedge 1 = 0 \\
 C_2 = P_2' \wedge X_1' \wedge X_2' \wedge X_3' = 1 \wedge 1 \wedge 0 \wedge 1 = 0
 \end{array}
 \right\}
 \longrightarrow (C_2C_1C_0)_{10} = 0
 \longrightarrow$$

هیچ یک از بیت ها دچار خطا نشده است.

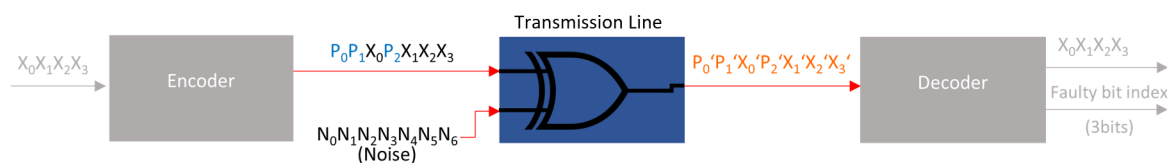


شکل ۴-۱۵ چند مثال از ورودی و خروجی های دریافتی دیکودر

برای انجام تست های بیشتر از **جدول بخش ۴** کمک بگیرید. برای تست ورودی های خراب، یک بیت از عدد ۷ بیتی جدول را قرینه کنید و به عنوان ورودی به ماژول خود بدهید و ببینید آیا ۴ بیت دیتا و شماره بیت خراب به درستی پیدا می شود یا خیر.

⚠️ تاکید می شود که در این روش، تنها می توان یک بیت معیوب را شناسایی و اصلاح کرد، بنابراین از خراب کردن همزمان بیش از ۱ بیت خودداری کنید!

۳-۶ بخش ششم (Transmission Line) (زمان تقریبی: ۳۰ دقیقه)



شکل ۴-۱۶ مدار Transmission line

در این بخش به طراحی transmission line می پردازیم که قرار است نقش رابط بین encoder و decoder را ایفا کند. این ماژول در ورودی دو عدد ۷ بیتی دریافت می کند و آن ها را به صورت bitwise با یکدیگر xor می کند. تا زمانی که N_0 تا N_6 همگی صفر باشند، حاصل xor آن با ورودی دیگر، همان ورودی دیگر خواهد بود. به محض اینکه یکی از بیت های N_0 تا N_6 یک شوند، آن بیت از ورودی دیگر به صورت قرینه در خروجی قرار خواهد گرفت. در حقیقت گیت xor یک inverter قابل برنامه ریزی است به این گونه که با یک کردن بیت n ام در ورودی دوم، خروجی برابر با ورودی اولی خواهد بود که بیت n ام اش قرینه شده است. به این شکل یک خطای ساختگی بر روی داده ی خروجی encoder را شبیه سازی می کنیم. حال برای تست ماژول transmission line و دیگر ماژول ها در کنار هم، آن ها را به شکل زیر به هم متصل کنید:



شکل ۴-۱۷ مدار کامل متشکل از ۳ ماژول بخش های قبل

یک عدد چهار بیتی توسط Dip4-Dip1 به ماژول encoder وارد می شود. جهت شبیه سازی وقوع خطا در انتقال اطلاعات، هفت بیت خروجی encoder به عنوان ورودی اول Transmission line، با هفت بیت {Dip5, Dip6, Dip7, Key0, Key1, Key2, Key3} به عنوان نویز و ورودی دوم، xor می شود. بدین ترتیب با فشردن هر یک از این کلیدها، بیت مربوطه بترتیب از صفر و یک به یک و صفر تغییر وضعیت داده و گویی در انتقال آن خطایی رخ داده است. اکنون این هفت بیت پس از xor شدن وارد decoder می شوند و چهار بیت اصلی کد را بر روی LED4-LED1 برد کمکی و شماره بیت معیوب را بر روی Dig0 سون سگمنت نمایش داده می شود.

بدین ترتیب، عدد معادل چهار کلید ورودی باید عیناً به LED4-LED1 مورد کمکی منتقل گردد. در آغاز تست می بایست عدد صفر بر روی Dig0 سون سگمنت نمایان شود، چرا که خطایی رخ نداده است. چنانچه یکی از کلیدهای ست شده جهت ایجاد خطا فشرده شوند، دیتای نمایش داده شده بر روی LED4-LED1 می بایست ثابت بماند و تنها عدد نمایان بر روی Dig0 سون سگمنت به شماره بیت معیوب شده تغییر کند. (منظور از Key ها کلیدهای فشاری موجود بر روی برد اصلی است، دقت داشته باشید که این کلیدها "Active LOW" هستند و چنانچه قرینه نشوند، در حالتی که هیچ دکمه‌ای فشرده نشده است، ۴ خطا ایجاد می کنند که اصلاً مطلوب نیست)

۳-۷ بخش هفتم (امتیازی)

می دانیم که روش Hamming code قادر به تشخیص و اصلاح تنها یک بیت معیوب در دیتای ارسال شده است. اما این روش همچنان می تواند وجود دو بیت معیوب را شناسایی کند ولی قادر به پیدا کردن مکان آن ها در دیتای ارسالی نیست فلذا امکان اصلاح آن را ندارد. بررسی کنید که از چه طریق می توان به وجود دو بیت معیوب پی برد و با تغییر ۳ مازول قبلی، سناریوی زیر را پیاده سازی کنید:

- در صورتی که بیت معیوبی وجود نداشت، ۴ بیت دیتا همچنان بر روی LED4-LED1 مورد کمکی و عدد صفر بر روی Dig0 سون گمنت ها نمایش داده شود.
 - در صورت وجود یک بیت معیوب، ۴ بیت دیتا بدون تغییر نسبت به بیت های ورودی بر روی LED4-LED1 و نمایش داده شود و شماره بیت معیوب بر روی Dig0 سون سگمنت نمایش داده شود.
 - در صورت وجود دو بیت معیوب، با توجه به اینکه امکان اصلاح وجود ندارد، حرف "E" یا همان عدد ۱۴ Hex بر روی Dig0 و Dig1 به معنای وجود دو ارور نمایان شده و LED4-LED1 همگی روشن شوند.
- در صورت نیاز به کلیدهای بیشتر، در استفاده از باقی دکمه های برد اصلی/کمکی آزاد هستید.

۴- گزارش

- مزیت استفاده از parity ها چیست که حاضر به ارسال ۳ بیت اضافه بر ۴ بیت داده هستیم؟
- چطور به روش hamming code قابل به تشخیص وجود دو خطا خواهیم بود؟ چرا در این حالت قادر به پیدا کردن مکان بیت های معیوب نمی باشیم؟
- در خصوص آنچه از این آزمایش آموختید گزارش ۱ الی ۲ صفحه‌ای (مطابق **تمپلیت** ارجاع شده در ابتدای این داکيومنت) بنویسید.
- دیدگاه خود نسبت به بخش‌های مختلف آزمایش را به انضمام پیشنهادهایتان بنویسید.